Generator of random numbers

The goal of these two exercise sessions is to implement some generators of random numbers, and investigate their behavior in terms of accuracy and convergence.

1 Implementing random generators

A large list of generators is proposed, their implementation can be selected with respect to your own interest or previous experiences.

1.1 Random generators based on Monte Carlo method

Linear congruential generator for uniform distribution on $]0;1[$

1. Implement a linear congruential generator (LCG) of uniform random numbers distributed on $[0;1]$.
   The implementation of the linear congruential generator is as follows:

$$\begin{cases} u_0 = \text{seed} \\ u_i = (ax_{i-1} + c) \bmod m, \qquad \forall i \in [1;N] \end{cases} \tag{1}$$

   where a is called the multiplier, c is called the increment and m is called the modulus. More details available in *Random Number Generation and Monte Carlo Methods*, James E. Gentle, p.11-14.

Multiplicative congruential generator for uniform distribution on $]0;1[$

2. How to change your previous code to implement a multiplicative congruential generator with for example $m = 2^{13} - 1$ and $a = 17$.

3. Modify your program to implement a matrix congruential generator:

$$\begin{bmatrix} x_{1i} \\ x_{2i} \end{bmatrix} \equiv \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_{1,i-i} \\ x_{2,i-1} \end{bmatrix} \mod m$$

   with $m = 2^{13} - 1$, $a_{11} = 17$, $a_{22} = 85$ and a_{12} and a_{21} variable. Choose some values for a_{12} and a_{22} between 0 and 17.

1.2 Generators based on a Latin Hypercube sampling

4. Implement a Latin hypercube sampling of Gaussian distributed independent random variables, or independant random vectors.

Remark: These samples could also be provided by the Matlab functions: lhsdesign and lhsnorm.

5. How to change your algorithm to provide an orthogonal Latin hypercube sampling of Gaussian distributed independant random vectors?

1.3 Generators based on Quasi Random sequences

Sobol sequence

6. We could implement a Sobol generator for uniform distribution of one-dimensional vector on $[0;1]$, or for two-dimensional random vectors on $[0;1]^2$. But, for this session, we will use the Sobol samples which are provided by the Matlab function sobolset.

Halton sequence

7. We could implement a Halton generator for uniform distribution of one-dimensional vector on $[0;1]$, or for two-dimensional random vectors on $[0;1]^2$. But, for this session, we will use the Halton samples which are provided by the Matlab function haltonset.

1.4 Generators for other distributions

Based on the inverse CDF method

8. Prove that if X is a random variable with an absolutely continuous distribution function F_X, the random variable $F_X(X)$ has a uniform distribution on $]0;1[$.

9. Implement a generator of a set of uniform random numbers on $]a;b[$ from a uniform set of data.

10. Implement a generator of a set of exponential random numbers on $[0;+\infty[$ from a uniform set of data.

Using Matlab

11. Generate a set of Gaussian random numbers using the function normrnd.

12. Generate a set of log-normal, and Weibull random numbers.

2 Investigate the repeatability of the generators

13. How to provide a repeatability of the sample of random variables, using the different generators previously implemented or used?

14. At the contrary, how to ensure to get different random sets?

3 Investigate the accuracy of the generators

In order to investigate their accuracy, we will not propose here to analyze mathematically the algorithm used by the generators. We will only observe some streams of numbers produced by the different generators, and investigate how their properties are closed to the expected properties.

15. To which properties can we refer to investigate this accuracy?

16. For one or several of these properties, investigate the accuracy of one of the previous generators. For example, investigate the lattice structure or the correlation of the pairs (x_n, x_{n+1}) for a multiplicative congruential method depending on the value of the multiplier.

4 Investigate the convergence of the generators

17. To investigate the convergence of a generator, we need to refer to a quantity of interest. Which quantities of interest could you consider to investigate the convergence of a generator?

18. Choose one generator, and investigate the convergence behavior of this generator with respect to some quantities of interest. For example, investigate the lattice structure of a two-dimensional Sobol generator.

19. Does the convergence behavior satisfy the results found in the literature?

Solution

1 Implementing some random generators

1.1 Random generators based on Monte Carlo method

Linear congruential generator for uniform distribution on $]0;1[$

1. Implementation of the linear congruential generator

$$\begin{cases} z_0 = \text{seed} \\ z_i = (az_{i-1} + c) \bmod m, \qquad \forall \;\; i \in [1;N] \end{cases} \tag{2}$$

Note that the implementation

a is called the multiplier, c is called the increment and m is called the modulus. More details available in *Random Number Generation and Monte Carlo Methods*, James E. Gentle, p.11-14.

Matlab file: `MCS.m`

```
seed = 46*1e5;

a = 10e3 ;
c = 0 ;
m = 2^31 - 1 ;

%a=3; c=4; m=3;

z = ones(dim,samp);

z(1,1) = seed;

if size(z,1) > 1
    for j = 2:size(z,1)
        z(j,1) = seed + j*(j+1)*seed;
    end
end

for i=2:size(z,2)
    z(:,i) = mod(a*z(:,i-1)+c,m) ;
end

x = z/m;
```

Multiplicative congruential generator for uniform distribution on $]0;1[$

2. How to change your previous code to implement a multiplicative congruential generator with for example $m = 2^{13} - 1$ and $a = 17$.

Matlab file: `mult_congruent.m`

A multiplicative congruential generator is a linear congruential where the increment c is equal to 0. More details available in *Random Number Generation and Monte Carlo Methods*, James E. Gentle, p.11-14.

3. Modify your program to implement a matrix congruential method:

$$\begin{bmatrix} x_{1i} \\ x_{2i} \end{bmatrix} \equiv \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_{1,i-i} \\ x_{2,i-1} \end{bmatrix} \mod m$$

with $m = 2^{13} - 1$, $a_{11} = 17$, $a_{22} = 85$ and a_{12} and a_{21} variable. Chose some values for a_{12} and a_{22} between 0 and 17.

Matlab file: `mult_mat_congruent.m`

```
a11 = 17;
a22 = 85;
a12 = 3;
a21 = 5;
c = 0 ;
m = 2^13 - 1 ;

z = ones(2,N);
z(1,1) = seed;
z(2,1) = seed;

for i=2:size(z,2)
    z(1,i) = mod(a11*z(1,i-1) + a12*z(1,i-1) + c,m) ;
    z(2,i) = mod(a21*z(2,i-1) + a22*z(2,i-1) + c,m) ;
end

u = z/m;
```

More details in Exercise 1.6 p.58 *Random Number Generation and Monte Carlo Methods*, James E. Gentle. Note that if the matrix $A = a_{ij}$ is diagonal, then the matrix generator is a set of scalar generators with different multipliers. For general non-diagonal A, the elements of the vectors are correlated.

1.2 Generators based on a Latin Hypercube sampling

4. Implement a Latin hypercube sampling of Gaussian distributed independent random variables, or independant random vectors.

Here, we present a Latin hypercube sampling based on discrete sets using quantile function. Another approach would be to compute integrals from the analytical expressions of the probability density function.

Matlab file: `LHS.m`

```
N=5e3;
rng('default')
sigma2 = 1050;
mu2 = 21000.0 ;
a = normrnd(mu2,sigma2,1,N);

for d =1:dim
% 1 - Range of known CDF is divided into N (number of samples) disjoint intervals
```

```
b = quantile(a,(1/samp:1/samp:(1-1/samp)));

% 2 - One value of each interval Skn is chosen randomly
for i = 1 : length(b)-1
        X(d,i) = random ('Uniform',b(i),b(i+1));
end
end
```

We could compare this function with the lhsdesign function provided by Matlab.
See functions norminv and lhsnorm provided by Matlab.

5. How to change your algorithm to provide an orthogonal Latin hypercube sampling of Gaussian distributed independant random vectors?

For the orthogonal Latin hypercube, the multi-variate integrals have to to been computed so that the multi-variate space is divided into equi-probable subspaces. Here, using the Matlab function quantile, this does not present any specific difficulty.

1.3 Generators based on Quasi Random sequences

Sobol sequence

6. We could implement a Sobol generator for uniform distribution of one-dimensional vector on $[0;1]$, or for two-dimensional random vectors on $[0;1]^2$. But, for this session, we will use the Sobol samples which are provided by the Matlab function sobolset.

See Matlab Sobol help page to know how to use it:

```
Generate a 3-D Sobol point set, skip the first 1000 values, and then retain every 101st point:

p = sobolset(3,'Skip',1e3,'Leap',1e2)
p =
    Sobol point set in 3 dimensions (8.918019e+013 points)
    Properties:
              Skip : 1000
              Leap : 100
    ScrambleMethod : none
        PointOrder : standard

Use scramble to apply a random linear scramble combined with a random digital shift:

p = scramble(p,'MatousekAffineOwen')
p =
    Sobol point set in 3 dimensions (8.918019e+013 points)
    Properties:
              Skip : 1000
              Leap : 100
    ScrambleMethod : MatousekAffineOwen
        PointOrder : standard

Use net to generate the first four points:

X0 = net(p,4)
X0 =
    0.7601    0.5919    0.9529
```

```
    0.1795    0.0856    0.0491
    0.5488    0.0785    0.8483
    0.3882    0.8771    0.8755

Use parenthesis indexing to generate every third point, up to the 11th point:

X = p(1:3:11,:)
X =
    0.7601    0.5919    0.9529
    0.3882    0.8771    0.8755
    0.6905    0.4951    0.8464
    0.1955    0.5679    0.3192
```

Halton sequence

7. We could implement a Halton generator for uniform distribution of one-dimensional vector on $[0;1]$, or for two-dimensional random vectors on $[0;1]^2$. But, for this session, we will use the Halton samples which are provided by the Matlab function haltonset.

See Matlab Halton help page to know how to use it:

```
Generate a 3-D Halton point set, skip the first 1000 values, and then retain every 101st point:

p = haltonset(3,'Skip',1e3,'Leap',1e2)

p =
    Halton point set in 3 dimensions (8.918019e+013 points)

    Properties:
              Skip : 1000
              Leap : 100
    ScrambleMethod : none

Use scramble to apply reverse-radix scrambling:

p = scramble(p,'RR2')

p =
    Halton point set in 3 dimensions (8.918019e+013 points)
    Properties:
              Skip : 1000
              Leap : 100
    ScrambleMethod : RR2

Use net to generate the first four points:

X0 = net(p,4)
X0 =
    0.0928    0.6950    0.0029
    0.6958    0.2958    0.8269
    0.3013    0.6497    0.4141
    0.9087    0.7883    0.2166

Use parenthesis indexing to generate every third point, up to the 11th point:
```

```
X = p(1:3:11,:)
X =
    0.0928    0.6950    0.0029
    0.9087    0.7883    0.2166
    0.3843    0.9840    0.9878
    0.6831    0.7357    0.7923
```

1.4 Generators for other distributions

Based on the inverse CDF method

8. Prove that if X is a random variable with an absolutely continuous distribution function F_X, the random variable $F_X(X)$ has a uniform distribution on $]0;1[$.

   X is a random variable with an absolutely continuous distribution function F_X. Let Y be the random variable $F_X(X)$. Then, for $t \in [0;1]$,

$$\begin{aligned} \Pr(Y \leqslant t) &= \Pr(F_X(X) \leqslant t) \\ &= F_X\left(F_X^{-1}(t)\right) \\ &= t, \end{aligned}$$

   assuming that the inverse function F_X^{-1} exists.

9. Implement a generator of a set of uniform random numbers on $]a;b[$ from a uniform set of data.

   The random variable U is uniformely distributed on $[0;1]$. If $U = F_X(X)$ where F_X is the cumulative distribution function of the desired distribution, F_X is continuous and strictly positive on $\mathbb{R}$. Thus, from a uniform sampling of N U_i, we can generate a set of N random variables X_i satisfying the cumulative distribution function F_X as:

$$X_i = F_X^{-1}\{U_i\}, \qquad \forall i \in [1;N]. \tag{3}$$

   For the uniform law, we can show that, $\forall X_i \in]a;b[$:

$$\begin{array}{rcccl} F_X(X_i) & = & \dfrac{X_i - a}{b-a} & = & U_i \\ F_X^{-1}(U_i) & = & X_i & = & a + (b-a)\,U_i. \end{array} \tag{4}$$

   Matlab file: `uni_ab_gen.m`

```
X = a + U*(b-a);
```

10. Implement a generator of a set of exponential random numbers on $[0;+\infty[$ from a uniform set of data.

    For the exponential law, we can show that:

$$\begin{array}{rcccl} F_X(X_i) & = & 1 - \exp -\lambda X_i & = & U_i \\ & = & \exp -\lambda X_i & = & 1 - U_i \\ F_X^{-1}(U_i) & = & X_i & = & -\dfrac{1}{\lambda} \ln 1 - U_i. \end{array} \tag{5}$$

    Matlab file: `exp_gen.m`

```
X = -log(1-U)/lambda;
```

Using Matlab

Probably already done during Session 1.

11. Generate a set of Gaussian random numbers using the function normrnd.
    Normal distribution:

    ```
    n = normrnd(25,5,[1 10]
    ```

12. Generate a set of log-normal, and Weibull random numbers.
    Log-normal:

    ```
    w = lognrnd(25,5,[1,10])
    ```

    Weibull distribution:

    ```
    w = 20 + 10 * wblrnd(0.5,0.4,[1 10])
    ```

2 Investigate the repeatability of the generators

13. How to provide a repeatability of the sample of random variables, using the different generators previously implemented or used?

We can repeat exactly the same set of random variables, by fixing the seed:

```
rand_gen(seed,N)
```

or for the rand function:

```
rng(seed)
```

14. At the contrary, how to ensure to get different random sets?

At the contrary to ensure to get a new set of random data, the seed is fixed with the time of the computer:

```
rand_gen(sum(100 x clock),N)
```

or for the function rand:

```
rng(sum(100 x clock))
```

See the two documents *New ways with random numbers.*

3 Investigate the accuracy of the generators

15. To investigate the convergence of a generator, we need to refer to a quantity of interest. Which quantities of interest could you consider to investigate the convergence of a generator?

We can investigate:

- the histogramm,
- the probability distribution,
- the cumulative distribution function,

- the moments of different orders,
- the lattice structure,
- the entropies,
- the chi-squared test,
- the correlation and covariance estimations of different lags.

16. For one or several of these properties, investigate the accuracy of one of the previous generators.

Histogramm

To examine the samples, we can represent their **histogramm**:

```
[nelements,center]=hist(Z,nb_hist),
```

or

```
histogram(Z,nb_hist).
```

Investigating Matlab functions

```
function [X] = rand_gauss()
X = normrnd(myMean,sqrt(var),1,N);
[nelem,center]=hist(X,30);
hold off
aa=-1.0:0.01:2.00;
bb = 1/sqrt(2*pi*power(sqrt(var),2)) * exp(-power((aa-myMean),2)/(2*power(sqrt(var),2)));
plot(aa,bb,'r');
hold on
plot(center,nelem/(sum(nelem)*(center(2)-center(1))),'b');
```

• Generate a set of Gaussian, log-normal, and Weibull random numbers. Represent their histogramm distribution:

```
hist(z) ; hist(z2) ;
```

Gaussian law:

```
Y = normrnd(des_mean, des_dev, 1, nb_samp); hist(Y)
```

Log-normal law:

```
Z = random('logn',100,0.5,1000,1); hist(Z,520)
```

Weibull law:

```
Z = random('Weibull',1.3,0.5,1000,1); hist(Z,800)
```

Probability density function

Plot their **probability density functions**:

$$f_X = \lim_{x_2 - x_1 \to 0} \frac{P(x \in [x_1; x_2])}{x_2 - x_1}$$

Matlab file: `getCdfPdf.m` or Matlab function `ksdensity`.

Compare Monte Carlo and LHS sampling using the probability density function(Result see Figure 1)

```
% Plot gaussian LHS sampling
[nelem,center]=hist(random_data2,30);

% PLot gaussian Monte-Carlo sampling
X = normrnd(mu2,sigma2,1,N);
[nelem2,center2]=hist(X,30);
hold off

% Plot Gaussian Law
aa=16000:500:26000;
bb = 1/sqrt(2*pi*power(sigma2,2)) * exp(-power((aa-mu2),2)/(2*power(sigma2,2)));

plot(aa,bb,'r','linewidth',2);
hold on
plot(center2,nelem2/(sum(nelem2)*(center2(2)-center2(1))),'g','linewidth',2)
plot(center,nelem/(sum(nelem)*(center(2)-center(1))),'b','linewidth',2)

legend('Gaussian Law','Monte Carlo sampling','LHS sampling')
legend('boxoff')
axis([16000 26000 0 5e-4])
set(gca,'FontSize',30,'fontName','Times');
xlabel('Young modulus [MPa]')
ylabel('f_{X}(x)')
```

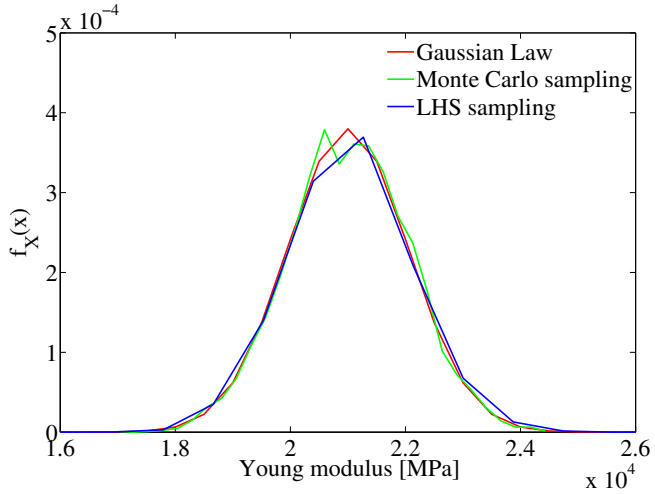


Figure 1: Comparison of Monte Carlo and LHS sampling

Cumulative distribution function

Plot their **cumulative ditribution function**:
Matlab functions:

```
cdfplot(z);
cdfplot(z2);
[F,x] = ecdf(z2)
```

or Matlab file: `getCdfPdf.m`

Moments of different orders

Specifying the mean value and the standard deviation of the law. Compute the true mean value and the true standard deviation of the generated sample.

Gaussian distribution • Generate a set of Gaussian random numbers using the function normrnd specifying the mean value and the standard deviation of the law. Compute the true mean value and the true standard deviation of the generated sample.

```
% error between the desired mean and the mean of the random data

function [error_mean]= error_gauss_gen(nb_samp)

% Gaussian random data generator
des_mean = 100 ;              % desired mean
des_dev = 0.5 ;              % desired standard deviation = Variance**0.5

Y = normrnd(des_mean, des_dev, 1, nb_samp);

true_mean = Mean(Y);
true_dev = sqrt(Variance(Y));

% disp('des_mean='); disp(des_mean);
% disp('true_mean='); disp(true_mean);
% disp('des_dev='); disp(des_dev);
% disp('true_dev='); disp(true_dev);

error_mean = abs(des_mean - true_mean) ;
error_dev = abs(des_dev - true_dev) ;

% error_mean = sqrt(des_mean^2 - true_mean^2) ;
% error_dev = sqrt(des_dev^2 - true_dev^2) ;

end
```

Evaluate the errors between the desired mean value, the desired variance and respectively the true mean value and the true variance.

```
% 28/01/2015
% Evaluate a generator

function []= eval_gen()

nb_test=1500;

for i=1:nb_test
    z = rand_gen(1,i);
    % z = rand(1,i);
```

```
    z2 = exp_gen(z,15);
    z3 = random('exp',1/15,1,nb_test);

    N(i)=i;
    the_mean(i) = Mean(z);
    the_var(i) = Variance(z);
    the_skew(i) = Skewness(z);
    the_kurt(i) = Kurtosis(z);
%
    the_mean2(i) = Mean(z2);
    the_var2(i) = var(z2);
    the_skew2(i) = Skewness(z2);
    the_kurt2(i) = Kurtosis(z2);
end
```

We can see that for values superior to 1312, the behavior of the exponential generator based on Linear Congruential Method is not satisfactory, concerning the variance, the skewness and the kurtosis. If we use the uniform random generator provided by Matlab, this problem is solved. This problem can also be tackled by changing the parameters of the LCG, $m = 2^{10} - 1$, for instance. But, anyway, the linear congruential generator is not suitable for Monte Carlo simulations because amongst other problems, its serial correlation (cross correlation of the signal with itself). See papers for more details.

Lattice structure

Periodicity of the sample We can investigate the periodicity of the Linear Congruential Generator.

Implement a space-filling process on $[0;1]^2$ with the three different random generators. Compare the three different sampling.

```
% 09/02/2015
% TD3: Random generator
% Compare Monte Carlo, LHS and QMC sampling
% for uniform distribution on [0,1]2

function[]=compare(nb_data)

random_data_MC = random('Uniform',0,1,nb_data,2);
random_data_lhs = lhsdesign(nb_data,2);
random_data_sobol = net(sobolset(2),nb_data);

figure
scatter(random_data_MC(:,1),random_data_MC(:,2));
title(['Monte Carlo - N = ', num2str(nb_data)],'Fontsize',16,'Fontweight','Bold')
set(gca,'Fontsize',16,'fontweight','Bold')
box on
grid on

figure
scatter(random_data_lhs(:,1),random_data_lhs(:,2));
title(['LHS - N = ', num2str(nb_data)],'Fontsize',16,'Fontweight','Bold')
set(gca,'Fontsize',16,'fontweight','Bold')
box on
grid on

figure
scatter(random_data_sobol(:,1),random_data_sobol(:,2));
title(['Sobol sequences - N = ', num2str(nb_data)],'Fontsize',16,'Fontweight','Bold')
```

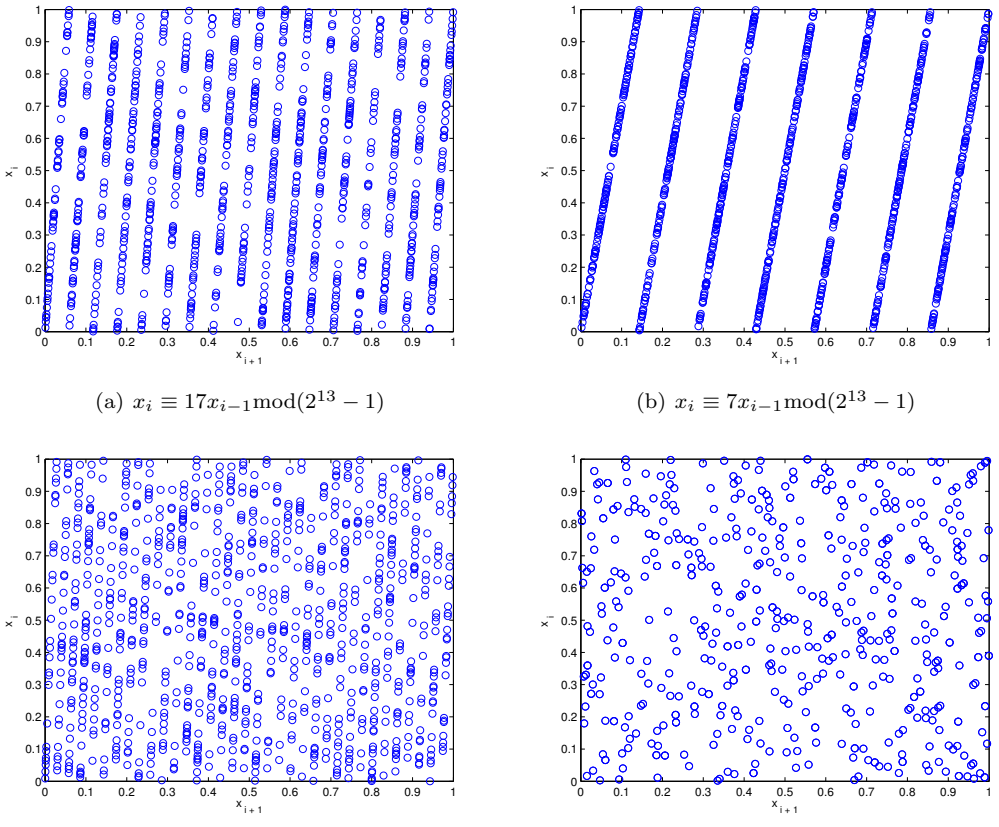
```
set(gca,'Fontsize',16,'fontweight','Bold')
box on
grid on
```

Investigation of the pairs (x_n, x_{n+1})

For multiplicative congruential method: Exercise 1.4 p.56 *Random Number Generation and Monte Carlo Methods*, James E. Gentle,
Generate 500 numbers x_i. Plot the pairs. On how many lines do the pairs lie ? Now, let $a = 85$. Generate 500 numbers, plot the pairs. Now look at the pairs x_{i+2} and x_i.

```
figure;
plot(u(1:length(u)-1), u(2:length(u)),'LineStyle','none','Marker','o')
xlabel('x_{ i + 1}');
ylabel('x_{ i }');
```

On how many lines do the pairs lie ? See Figure 2. The pairs lie on a lines, 17 lines for $a = 17$, 85 lines for $a = 85$.



(a) $x_i \equiv 17x_{i-1} \mathrm{mod}(2^{13} - 1)$

(b) $x_i \equiv 7x_{i-1} \mathrm{mod}(2^{13} - 1)$

(c) $x_i \equiv 35x_{i-1} \mathrm{mod}(2^{13} - 1)$

(d) $x_i \equiv 1000x_{i-1} \mathrm{mod}(2^{13} - 1)$

Figure 2: Pairs of Succesive Numbers from some multiplicative congruential generators

Entropies

See attached paper: p.68, *Random Number Generation and Monte Carlo Methods*, James E. Gentle.

Chi-squared test

Test: chi-squared tests for uniformity in any dimension, chi-squared tests for triangularity of sums of two successive numbers, and so on.
See attached paper: p.69, *Random Number Generation and Monte Carlo Methods*, James E. Gentle.

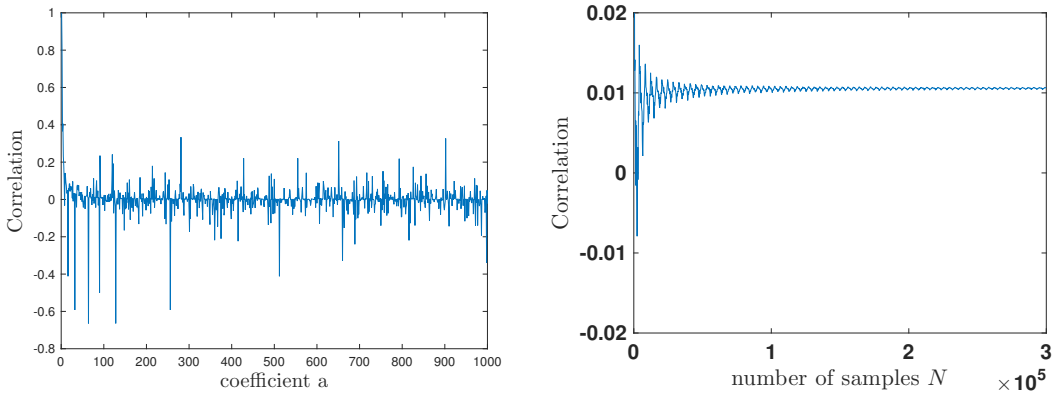
Correlation and covariance of different lags

Tests for serial correlation of various lags and various sign tests are other possibilities.

For multiplicative congruential method: Exercise 1.4 p.56 *Random Number Generation and Monte Carlo Methods*, James E. Gentle,
Generate 500 numbers x_i. Compute the correlation of the pairs of successive numbers x_{i+1} and x_i. Now, let $a = 85$. Generate 500 numbers, compute the correlation of the pairs. Now look at the pairs x_{i+2} and x_i. Compute their correlation

```
X = u(1:length(u)-1);
Y = u(2:length(u));
C = cov(X,Y,1)/(sqrt(var(X)*var(Y)));
```

The covariance matrix is symmetric. The diagonal of the matrix contains the variance of each random component. For more specific investigation on covariance matrix, refer to Session 1 & 2, Exercise 5.
With $a = 17$, the correlation of pairs of successive numbers should be about 0.09. With $a = 85$, the correlation of lag 1 is about 0.03, but the correlation of lag 2 is about -0.09 (???).



(a) Influence of coefficient a

(b) Influence of the number of samples

Figure 3: Serial Correlation

For multiplicative matrix congruential method: Exercise 1.5 p.57 *Random Number Generation and Monte Carlo Methods*, James E. Gentle,
Letting a_{12} and a_{22} vary between 0 and 17. Generate 500 vectors and compute their sample variance-covariances. Are the variances of the two elements in your vectors constant? Explain. What about the covariances ? Can

you see any relationship between the covariances and a_{12} and a_{21}? Is there any reason to vary a_{12} and a_{21} separately? Can a lower triangular matrix (that is, one with $a_{21} = 0$) provide all of the flexibility of matrices with varying values of a_{21} ?

Applications

With respect to all these properties, we can compare the Monte Carlo, LHS and QMC samplings.
For the application of the investigation of the accuracy of the generator, we could investigate:

- some problems of the multiplicative congruential generator when the multiplier is too close to the modulus - Exercise 1.5 p.57 *Random Number Generation and Monte Carlo Methods*, James E. Gentle,

4 Investigate the convergence of the generators

17. To investigate the convergence of a generator, we need to refer to a quantity of interest. Which quantities of interest could you consider to investigate the convergence of a generator?

We can investigate the convergence behavior of the generator with respect to:

- any stochastic moment of the sample: the mean, the variance, the skewness, the kurtosis.
- the histogramm distribution,
- the probability distribution,
- the cumulative distribution function,
- the lattice structure,
- the entropies,
- the chi-squared test estimation,
- the correlation and covariance estimations of different lags.

18. Choose one generator, and investigate the convergence behavior of this generator with respect to some quantities of interest.

Stochastic moments

We can, for example, plot the root mean square error between the expected value and the real value of one of the moments, and investigate how evolve these errors with respect to the number of samples:
Matlab file: `eval_gen.m`

```
function []= eval_gen()
nb_test=1500;
for i=1:nb_test
    z = rand_gen(1,i);
    z = rand(1,i);
    z2 = exp_gen(z,15);
    z4 = uni_ab_gen(z,50,300);
    z3 = random('exp',1/15,1,nb_test);
    N(i)=i;
    the_mean2(i) = Mean(z); the_var2(i) = Variance(z);
    the_skew2(i) = Skewness(z); the_kurt2(i) = Kurtosis(z);
```

```
end
plot(N,the_mean2);
hold on
plot(N,the_var); plot(N,the_var2,'g'); % plot(N,the_skew2,'r'); % plot(N,the_kurt2,'k');

function err_convergence
nb_samp = 10000;
samp = [1:nb_samp] ;
error = [];
for i=1:nb_samp
error(i) = error_gauss_gen(samp(i));
end
plot(samp,error)
```

Histogramm

To examine the samples, we can represent their **histogramm**:

```
[nelements,center]=hist(Z,nb_hist),
```

or

```
histogram(Z,nb_hist),
```

Probability density function

Plot their **probability density functions**:

$$f_X = \lim_{x_2 - x_1 \to 0} \frac{P(x \in [x_1; x_2])}{x_2 - x_1}$$

Matlab file: `getCdfPdf.m` or Matlab function `ksdensity`.

Cumulative distribution function

Plot their **cumulative ditribution function**:
Matlab functions:

```
cdfplot(z);
cdfplot(z2);
[F,x] = ecdf(z2)
```

or Matlab file: `getCdfPdf.m`

Lattice structure

Quasi Random sequences Implement a Sobol generator for uniform distribution of one-dimensional vector on $[0; 1]$. and of two-dimensional random vectors on $[0; 1]^2$. For this, we will use the Matlab functions set and sobolset: random_data = net(sobolset(dimension),nb_samples)

```
% 09/02/2015
% TD 3: random generators
% Sobol generator
% for uniform distribution
```

```
function[random_data] = Sobol(nb_data, dim)

% nb_data = Number of random data
% dim: dimensions of random data

if dim == 1
    random_data = net(sobolset(1),nb_data);
    figure
    y = ones(nb_data,1);
    plot(random_data,y,'rs')
    title(['N = ', num2str(nb_data)],'Fontsize',16,'Fontweight','Bold')
    set(gca,'Fontsize',16,'fontweight','Bold')
    set(gca,'YTickLabel',[])
    box on
    grid on
elseif dim ==2
    random_data = net(sobolset(2),nb_data);
    figure
    scatter(random_data(:,1),random_data(:,2));
    title(['Sobol sequences - N = ', num2str(nb_data)],'Fontsize',16,'Fontweight','Bold')
    set(gca,'Fontsize',16,'fontweight','Bold')
    box on
    grid on
else
    % Non implemented for dimensions larger than 2
end
```

Result see Figure 4: we can observe the strongly 'periodic' arrangement of Sobol sequences.

Entropies

See attached paper: p.68, *Random Number Generation and Monte Carlo Methods*, James E. Gentle.

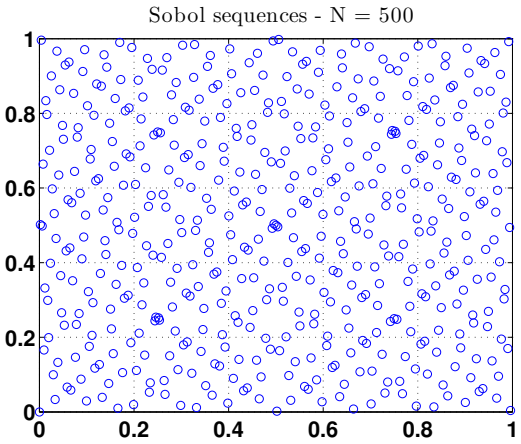
Chi-squared test

See attached paper: p.69, *Random Number Generation and Monte Carlo Methods*, James E. Gentle.

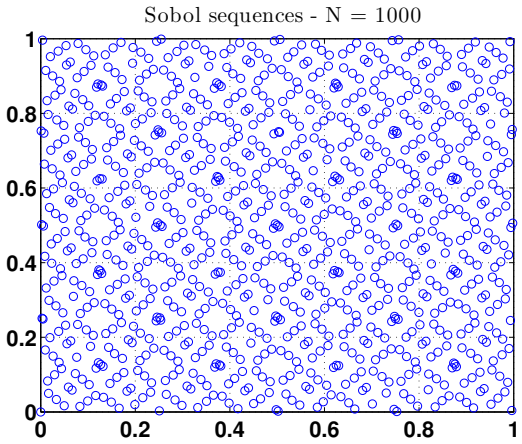
Correlation and covariance estimations of different lags

This can also be investigated see Figure 3 and the explanations p.14.

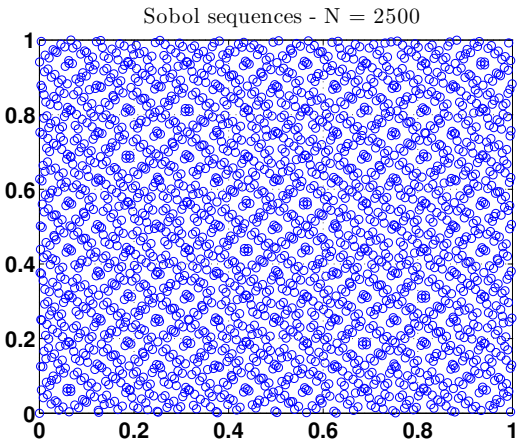
19. Does the convergence behavior satisfy the results provided by the literature?



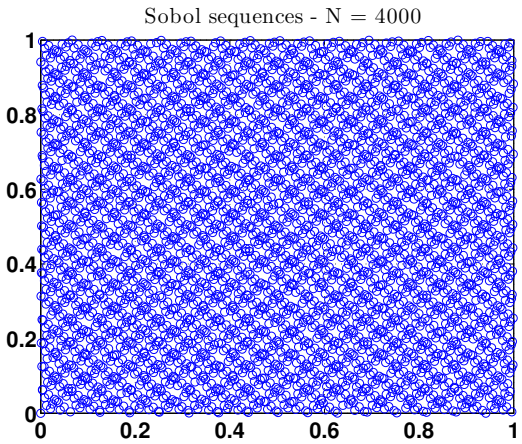
(a) Sample of 500 numbers



(b) Sample of 1000 numbers



(c) Sample of 2500 numbers



(d) Sample of 4000 numbers

Figure 4: Sample Numbers from a Sobol generators